FinMate: A Personal Financial Decision Support System Using Multi Agent Intelligence and Retrieval Augmented Generation

First Author, Second Author, Third Author, Fourth Author, Fifth Author
Department of Computer Science and Engineering
Institution Name
City, Country
email@example.com

***Abstract*—Personal financial management requires users to interpret transaction data, monitor expenditure, evaluate financial goals, and make informed decisions under uncertainty. Existing financial management applications primarily provide descriptive dashboards and rule based recommendations, while general purpose conversational systems may generate financial explanations without deterministic verification or domain grounded evidence. This paper presents FinMate, a personal financial decision support system that combines deterministic financial computation, machine learning, retrieval augmented generation, multi agent orchestration, and privacy focused mobile transaction ingestion. The system uses a FastAPI based backend with specialized agents for transaction analysis, budgeting, financial goals, investments, and decision synthesis. Transaction classification is performed using a TF-IDF and Linear Support Vector Classifier pipeline, expense forecasting uses Ridge Regression with lagged temporal features, and expenditure anomalies are detected using Isolation Forest. A semantic retrieval pipeline provides regulatory context from financial guidance sources including RBI, SEBI, and FPSB India. Monetary computations are isolated from language model generation and use Python Decimal arithmetic to maintain exact financial values. FinMate additionally implements a human in the loop governance mechanism for state changing operations and an Android based SMS ingestion pipeline that parses transaction alerts locally without transmitting raw SMS content. The implemented system reports 98.4 percent transaction classification accuracy, a macro F1 score of 0.9849, a 93.3 percent retrieval hit rate at three results, and 102 out of 102 automated tests passing.**

***Index Terms*—personal finance, financial decision support, machine learning, retrieval-augmented generation, multi-agent systems, large language models, financial data privacy, anomaly detection**

I. Introduction

Personal financial management involves a combination of transaction tracking, expenditure analysis, budgeting, savings planning, goal management, and investment related decision support. The increasing availability of digital financial records has created opportunities for automated analysis, but the usefulness of such systems depends on their ability to transform heterogeneous financial data into accurate, interpretable, and actionable information.

Conventional personal finance applications commonly emphasize transaction recording, dashboards, categorization, and historical summaries. Such functionality is useful for understanding past financial activity, but it does not necessarily provide an integrated mechanism for interpreting financial scenarios, comparing alternatives, forecasting expenditure, or grounding recommendations in authoritative financial guidance. At the same time, large language models (LLMs) provide natural language interaction and reasoning capabilities, but their direct use for financial computation introduces risks because generated numerical values may not correspond to authoritative transaction records or deterministic financial calculations.

FinMate addresses this problem through a layered financial decision support architecture that separates numerical computation from natural language generation. The system combines a deterministic financial calculation engine with machine learning models, retrieval augmented generation (RAG), and a collaborative multi agent orchestration layer. The architecture is implemented using Python, FastAPI, Pydantic, SQLAlchemy, PostgreSQL, and Scikit-learn, with a React based web workspace and a native Android client for privacy focused transaction ingestion.

The proposed system employs specialized financial agents for transaction analysis, budgeting, goal management, investment related educational analysis, and cross-domain decision synthesis. An orchestrator performs intent triage and controls agent execution through bounded tool access. Financial values used by the language model are obtained from backend calculation services rather than generated directly by the model. This separation is intended to reduce numerical inconsistencies and improve the traceability of financial responses.

FinMate also incorporates retrieval augmented generation for financial regulatory and planning knowledge. The implemented retrieval subsystem uses vector representations and cosine similarity to retrieve relevant content from a knowledge corpus covering RBI, SEBI, and FPSB India guidelines. Retrieved information can subsequently be supplied as context for natural language explanations.

Privacy is addressed at the transaction ingestion layer through the native Android FinMateSMS component. Bank transaction alerts are filtered and parsed locally using deter-

ministic regular expressions. The system uses sender whitelisting, OTP and secret-PIN filtering, local hashing, and a Room based transaction queue before synchronized transaction data is sent to the backend. Raw SMS content is not transmitted by the ingestion pipeline.

A further design objective is to maintain human control over operations that could modify financial records. FinMate classifies operations into read-only actions, simulations, and state-mutating actions. Mutating operations are staged through a human-in-the-loop approval mechanism and require explicit user confirmation before persistent state is modified.

The principal contributions of this work are as follows:

- A unified personal financial decision support architecture combining deterministic financial computation, machine learning, retrieval augmented generation, and multi agent orchestration.
- A separation between numerical financial computation and language model generation using exact decimal arithmetic and structured backend outputs.
- A governed multi agent architecture with specialized financial agents, constrained tool access, execution limits, and human approval for state changing operations.
- A regulatory retrieval pipeline based on semantic similarity over a financial knowledge corpus containing RBI, SEBI, and FPSB India guidance.
- A privacy focused Android transaction ingestion mechanism that performs SMS parsing locally and avoids transmission of raw SMS content.
- An implemented evaluation framework covering machine learning performance, retrieval performance, agent behavior, application functionality, and automated testing.

The remainder of this paper is organized as follows. Section II discusses related work and identifies the system gap addressed by FinMate. Section III defines the problem and design objectives. Section IV presents the proposed system architecture. Section V describes the methodology and implementation. Section VI discusses the machine learning, retrieval augmented generation, and language model components. Section VII presents the security and privacy architecture. Section VIII evaluates the implemented system. Section IX discusses the findings and limitations, followed by future work in Section X and the conclusion in Section XI.

II. Related Work

Artificial intelligence has increasingly been applied to personal financial management to support budgeting, expenditure analysis, forecasting, and decision making. Vaduka *et al.* studied AI-driven decision support for personal finance and identified applications across budgeting, investment, debt management, and retirement planning while also noting privacy and transparency as important deployment concerns [1]. Shailendra *et al.* presented Money Map, a data-driven personal finance tracker that combines machine learning, budget forecasting, visualization, and personalized financial insights [2]. Lakshmi *et al.* described an AI-powered financial consultant incorporating budget management, expense tracking, financial education, financial calculators, and predictive models for financial analysis [3]. These systems demonstrate the feasibility of AI-assisted personal finance, but they primarily emphasize integrated application functionality rather than the combination of deterministic financial computation, governed multi-agent execution, regulatory retrieval, and privacy-focused transaction ingestion considered in FinMate.

Transaction categorization is an established application of machine learning in financial software. Recent work has investigated automated classification of personal expenses from unstructured transaction descriptions using natural language processing and machine learning methods [4]. Related research has also explored transaction anomaly detection using supervised and unsupervised machine learning methods for identifying deviations in financial transactions [5], [6]. These studies support the use of machine learning for transaction-level analysis. FinMate extends this task within a broader decision-support pipeline in which classification and anomaly signals become inputs to downstream financial analysis rather than isolated prediction outputs.

Forecasting and predictive analytics are also increasingly used in personal financial applications. Recent research has investigated automated transaction classification together with budget prediction, showing the growing use of predictive models for personal financial planning [7]. FinMate similarly performs expenditure forecasting, but uses a strictly lagged feature construction strategy and separates forecast computation from language model narration. This separation allows financial values and uncertainty intervals to originate from backend model outputs rather than being generated directly by a language model.

Large language models have introduced natural language interfaces for complex information and decision-support tasks. However, standalone language models can suffer from factual hallucination and limited access to current or domain-specific knowledge. Retrieval-Augmented Generation (RAG) addresses these limitations by combining language generation with retrieval from an external knowledge source [8]. Recent surveys further describe the evolution of RAG toward more advanced and modular retrieval pipelines and emphasize the importance of grounding generated responses in verifiable external information [9]. These properties are particularly relevant to financial applications where regulatory guidance and other authoritative information should be distinguishable from generated explanations.

Multi-agent architectures provide another mechanism for decomposing complex tasks into specialized components. Surveys of multi-agent systems describe architectures in which agents coordinate through structured communication and workflow mechanisms [10]. More recent work on large model-based agents highlights planning, tool use, cooperation, security, and privacy as important considerations when building agentic systems [11]. A 2026 survey of LLM agents similarly identifies memory, planning, and tool use as central components while highlighting reliability and evaluation challenges [12]. FinMate applies these concepts to personal

financial decision support through domain-specific agents, a centralized orchestrator, bounded tool access, execution limits, and explicit human-in-the-loop approval for persistent state changes.

The literature therefore establishes several relevant capabilities: AI-based personal financial management, machine learning for transaction analysis, RAG for knowledge grounding, and multi-agent architectures for task decomposition. However, these capabilities are commonly studied as separate components. FinMate combines them in a single decision-support architecture and adds two implementation constraints that are central to the proposed system: monetary computations are performed by a deterministic financial engine rather than generated by the language model, and state-changing operations are subject to explicit human authorization. The Android ingestion component additionally performs transaction extraction locally from authorized bank SMS messages, supporting a privacy-oriented data collection workflow.

The resulting research gap is therefore not the invention of an individual machine learning model, RAG, or multi-agent architecture. Rather, the focus is the engineering and evaluation of their coordinated use for personal financial decision support under deterministic computation, regulatory grounding, privacy constraints, and human governance.

III. Problem Definition and Design Objectives

The objective of FinMate is to provide an intelligent financial decision support system capable of transforming heterogeneous personal financial records into transparent analytical information and scenario based decision support while maintaining deterministic numerical correctness and human control.

The system addresses the following requirements:

1) Accurate classification and organization of transaction data.
2) Forecasting of future expenditure from historical financial patterns.
3) Identification of anomalous expenditure behavior.
4) Retrieval of relevant financial and regulatory guidance.
5) Natural language interaction through large language models.
6) Deterministic computation of monetary values independent of language model generation.
7) Simulation of financial scenarios without modifying persistent records.
8) Human authorization for state changing operations.
9) Privacy preserving ingestion of transaction information.
10) Reliable storage, validation, lineage tracking, and duplicate prevention for financial records.

The resulting system is designed as a decision support platform rather than an autonomous financial agent. It does not independently execute trades, transfer funds, or modify financial records without verified human authorization.

IV. Proposed System Architecture

FinMate is designed as a layered personal financial decision-support architecture that combines client applications, a REST API gateway, multi-agent orchestration, machine learning and retrieval services, deterministic financial computation, data engineering, and persistent storage. The architecture is designed to ensure that natural language interaction does not replace authoritative financial computation and that state-changing operations remain subject to explicit user authorization.

A. System Overview

The system consists of two primary client interfaces: a React-based web workspace and a native Android client named FinMateSMS. The web workspace provides the principal interaction environment for financial analysis, agent execution traces, financial dashboards, scenario simulations, and human-in-the-loop approval. The Android client provides privacy-focused transaction ingestion by processing authorized bank SMS alerts locally.

Both clients communicate with the backend through the FastAPI REST API gateway. The backend exposes versioned endpoints under `/api/v1` and uses Pydantic v2 schemas for request and response validation. The gateway routes requests to the appropriate domain subsystem, including agent orchestration, artificial intelligence, transactions, goals, investments, decisions, and data ingestion.

The persistence layer uses SQLAlchemy 2.0 as the object-relational mapping layer. PostgreSQL 16 is the primary production database, while SQLite provides a resilient local fallback. Database migrations are managed through Alembic.

B. Client Layer

1) Web Workspace: The web workspace is implemented as a single-page application using React 19, TypeScript, Vite, Tailwind CSS v4, Lucide React, and Recharts. It provides several functional views:

- **AI Advisor Hub:** Provides interaction with the multi-agent financial decision-support subsystem and displays execution traces, facts, forecasts, trade-offs, and regulatory citations.
- **Human-in-the-Loop Review:** Displays proposed ledger modifications through visual difference cards and allows explicit approval or rejection.
- **Financial Dashboard:** Presents financial indicators including monthly surplus, cumulative savings, goal progress, and asset allocation.
- **CSV Ingestion Wizard:** Provides pre-commit validation, delimiter detection, row-level quality information, and import preview.
- **Transactions Ledger:** Provides searchable and filterable transaction records together with category and data-quality information.
- **Goals and Portfolio Views:** Presents financial milestones, time-horizon projections, and asset allocation information.

2) Native Android Client: The FinMateSMS Android client is implemented using Kotlin 1.9+ and Android SDK APIs 24–34. AndroidX Room provides local persistence, while WorkManager manages network-constrained background synchronization. Retrofit and OkHttp provide communication with the FastAPI backend.

The Android client does not transmit raw SMS messages. Instead, authorized bank messages are filtered and parsed locally. The resulting structured transaction information is stored in a Room database and subsequently synchronized with the backend when network connectivity is available.

C. FastAPI REST Gateway

The FastAPI gateway forms the central communication boundary between client applications and backend services. It uses Pydantic v2 for structured validation and provides versioned REST endpoints under `/api/v1`.

The gateway exposes interfaces for:

- multi-agent task submission and execution status;
- human-in-the-loop approval and rejection;
- transaction classification, forecasting, and anomaly detection;
- regulatory RAG queries;
- SMS-based transaction ingestion;
- transaction and financial profile management;
- CSV import preview and commitment;
- goal and investment retrieval;
- deterministic financial decision simulation and comparison.

This separation allows the client applications to remain presentation layers while domain logic, financial computation, artificial intelligence, and data validation remain within controlled backend services.

D. Multi-Agent Orchestration Layer

The multi-agent subsystem coordinates domain-specific financial agents through a centralized AI Orchestrator. The orchestrator performs intent triage, determines the appropriate execution path, enforces execution limits, and collects structured outputs from specialized agents.

The principal specialized agents are:

- **Transaction Specialist:** Analyzes spending patterns, transaction classifications, and expenditure anomalies.
- **Budget Specialist:** Evaluates cash flow, surplus, savings rates, and projected expenditure.
- **Goal Specialist:** Evaluates financial milestones, required contribution rates, and goal feasibility.
- **Investment Specialist:** Provides asset allocation analysis and investment-related educational information subject to risk disclosures and regulatory grounding.
- **Financial Decision Agent:** Synthesizes evidence from multiple specialists into structured decision alternatives and associated trade-offs.

The orchestrator uses bounded execution parameters including a maximum of eight agent steps, twelve tool calls, and three agent handoffs. This bounded execution model is intended to prevent uncontrolled recursive agent behavior.

E. Governed Tool Registry

FinMate provides a governed registry containing fifteen domain tools. Each tool has structured input and output contracts enforced through Pydantic schemas. Agent-specific allowlists restrict which tools each specialist can invoke.

The registry also applies tenant injection to ensure that operations are associated with the correct user context. This design follows a least-privilege approach in which an agent receives access only to tools required for its assigned domain.

The resulting execution path can be represented as

$$Q \rightarrow O \rightarrow A_i \rightarrow T_j \rightarrow R, \tag{1}$$

where Q represents the user query, O represents the orchestrator, A_i represents a selected specialist agent, T_j represents a permitted tool, and R represents the resulting structured response.

F. Machine Learning Layer

The machine learning subsystem contains three principal analytical components: transaction classification, expense forecasting, and anomaly detection.

Transaction descriptions are processed using a TF-IDF representation and a Linear Support Vector Classifier. The classifier maps descriptions to ten financial categories.

Expense forecasting uses Ridge Regression with strictly lagged temporal features. The feature set includes previous monthly expenditure values, three-month rolling averages, and month information.

Anomaly detection uses Isolation Forest to identify unusual expenditure observations without requiring labeled anomaly examples.

The machine learning subsystem provides structured analytical outputs to the decision-support layer rather than directly generating natural language responses.

G. Regulatory Retrieval Layer

FinMate implements a semantic retrieval pipeline for financial and regulatory knowledge. The knowledge corpus includes guidance associated with the Reserve Bank of India (RBI), Securities and Exchange Board of India (SEBI), and Financial Planning Standards Board India (FPSB India).

The retrieval process represents the query and knowledge entries in a vector space and evaluates their similarity using cosine similarity. The highest-scoring relevant entries are returned to the downstream language model as contextual evidence.

The retrieval component therefore provides the following flow:

$$q \rightarrow v_q \rightarrow \text{Similarity}(v_q, V_K) \rightarrow K_r \rightarrow L, \tag{2}$$

where q is the user query, v_q is its vector representation, V_K represents the indexed knowledge vectors, K_r represents

the retrieved knowledge, and L represents the downstream language model context.

H. Deterministic Financial Computation Layer

A dedicated financial computation engine performs monetary calculations independently of the language model. Python's `Decimal` type is used with `ROUND_HALF_UP` to avoid binary floating-point representation errors in financial values.

The principal calculations include monthly surplus, savings rate, goal contribution requirements, financial runway, and scenario-based changes to financial milestones.

For example, monthly surplus is computed as

$$S = I - (F + D), \tag{3}$$

where I represents monthly income, F represents fixed expenditure, D represents discretionary expenditure, and S represents monthly surplus.

The deterministic computation layer supplies numerical outputs to the decision-support and language-generation components. Consequently, the LLM is responsible for explanation and synthesis rather than independently calculating authoritative monetary values.

I. Counterfactual Decision Simulation

FinMate contains an in-memory what-if simulation sandbox for evaluating financial alternatives without modifying persistent records. The simulation engine calculates a baseline financial state and applies temporary deltas representing potential financial decisions.

Supported scenario categories include purchases, savings changes, expense changes, goal contributions, debt payments, investments, subscriptions, income changes, and custom scenarios.

The engine can produce multiple deterministic pathways, including immediate actions, deferred or staggered actions, and reallocation of discretionary expenditure. Each pathway can be evaluated using changes in monthly surplus, savings, financial buffer, and goal milestone timelines.

Because the simulation executes in memory, rejected or exploratory scenarios do not alter the persistent financial ledger.

J. Data Engineering and Quality Layer

FinMate includes an ingestion and data-quality subsystem for heterogeneous financial records. The CSV parser automatically identifies common delimiters and supported encodings. The normalizer handles different date formats, credit and debit representations, and currency symbols.

An eleven-rule data-quality engine evaluates imported records. Rules are classified into error, warning, and informational categories. Potential duplicates are identified using a sliding-window comparison involving user identity, transaction date, amount, and normalized description.

Transaction lineage is retained through fields including source type, source reference, import identifier, and data-quality flags. This enables the system to trace financial records back to their ingestion source.

K. Persistence Layer

The persistence layer is implemented using SQLAlchemy 2.0 and PostgreSQL 16. Core entities include users, financial profiles, transactions, categories, goals, investments, import records, agent tasks, agent approvals, and agent tool calls.

SQLite is supported as a resilient fallback for environments where a PostgreSQL instance is not available. Alembic is used for schema migration management.

L. End-to-End Data Flow

The complete FinMate processing pipeline can be summarized as follows:

1) A user submits a financial query or transaction through the web or Android client.
2) The request reaches the FastAPI REST gateway, where the payload is validated.
3) The orchestrator determines the appropriate financial domain and selects one or more specialized agents.
4) Agents invoke only the tools permitted by their domain allowlists.
5) Machine learning models, deterministic financial calculations, database services, and the regulatory retrieval engine provide structured evidence.
6) The Financial Decision Agent synthesizes the collected evidence into decision-support alternatives, facts, forecasts, and trade-offs.
7) If the requested operation is read-only, the result can be returned directly. Simulation operations execute only in memory.
8) If an operation would modify persistent financial state, the Human-in-the-Loop engine creates a pending approval record.
9) Persistent modification occurs only after explicit user confirmation.

This architecture establishes a separation between data acquisition, deterministic financial computation, predictive analytics, knowledge retrieval, language-based explanation, and state mutation. Such separation forms the basis for the security, privacy, and evaluation mechanisms described in subsequent sections.

V. Methodology and Implementation

FinMate follows a modular methodology in which financial data acquisition, validation, deterministic computation, machine learning, semantic retrieval, agent orchestration, and decision presentation are implemented as separate but coordinated subsystems. The design emphasizes a strict separation between numerical financial computation and natural language generation.

A. Financial Data Acquisition

FinMate supports multiple sources of financial information. Structured transaction records can be imported through the CSV ingestion pipeline, while the native Android FinMateSMS client can obtain transaction information from authorized bank SMS alerts.

The CSV ingestion pipeline first identifies the input delimiter and encoding. Supported delimiters include comma, semicolon, tab, and pipe formats. The normalization stage subsequently converts heterogeneous date representations and credit/debit conventions into the internal transaction schema.

The normalized transaction record contains information required for financial analysis while retaining source lineage. Each imported batch is associated with an import identifier, and individual records retain source metadata for subsequent auditing.

B. Data Quality and Validation

Before financial records are committed to persistent storage, FinMate applies a deterministic data-quality engine. The engine contains eleven validation rules distributed across three severity levels.

- **Error:** Identifies records that cannot be safely processed, including invalid dates, invalid amounts, missing mandatory identifiers, or invalid categories.
- **Warning:** Identifies potentially problematic records, including statistical anomalies, possible duplicates, and unclassified transactions.
- **Information:** Records successful enrichment operations, normalization events, and format conversions.

Records classified as errors are prevented from entering the persistent ledger until the associated issue has been resolved. Warnings are retained as data-quality metadata so that downstream financial analysis can account for potentially uncertain records.

C. Transaction Deduplication and Lineage

FinMate employs duplicate detection at both ingestion and transaction processing stages. For structured imports, a sliding-window heuristic compares user identity, transaction date, amount, and normalized description.

For Android SMS transactions, a SHA-256 hash is generated from the processed SMS information. The hash acts as a stable source reference. Android Room maintains a unique constraint on this value, while the backend performs an additional server-side duplicate check.

The dual-layer strategy can be represented as

$$D = \begin{cases} 1, & \text{if } h \in H_{\text{local}} \vee h \in H_{\text{server}}, \\ 0, & \text{otherwise}, \end{cases} \tag{4}$$

where h is the transaction hash, H_{local} is the set of locally stored hashes, and H_{server} is the set of hashes already recorded by the backend.

Transaction lineage is retained using source type, source reference, import identifier, and data-quality metadata. This allows analytical outputs to be traced to their underlying financial records.

D. Deterministic Monetary Computation

Financial arithmetic is intentionally separated from the language model. FinMate uses Python's `Decimal` data type with `ROUND_HALF_UP` rounding. Persistent monetary values use `NUMERIC(15,2)` database fields.

This design avoids the use of binary floating-point arithmetic for authoritative monetary calculations.

The monthly financial surplus is calculated as

$$S = I - (F + D), \tag{5}$$

where I is total monthly income, F represents fixed expenses, and D represents discretionary expenditure.

The savings rate is computed from savings contributions and total inflows:

$$R_s = \frac{C_s}{I_t} \times 100, \tag{6}$$

where C_s is the savings contribution and I_t is the total inflow during the corresponding period.

For a financial goal, the required monthly contribution is calculated as

$$C_m = \frac{T - C}{M}, \tag{7}$$

where T is the target amount, C is the current accumulated amount, and M is the remaining number of months.

These calculations are performed by backend services and are not delegated to the LLM.

E. Transaction Classification

FinMate automatically categorizes transaction descriptions using a supervised machine learning pipeline based on TF-IDF feature extraction and a Linear Support Vector Classifier.

The classifier maps transaction descriptions into ten categories:

1) Housing
2) Food and Dining
3) Transportation
4) Utilities
5) Healthcare
6) Entertainment
7) Shopping
8) Personal Care
9) Education
10) Investments

The textual transaction description is transformed into a TF-IDF feature vector and supplied to the trained classifier. The resulting category is stored as structured transaction metadata and can subsequently be used by the budget, anomaly, and decision-support components.

The implemented system reports an accuracy of 98.4 percent and a macro F1-score of 0.9849 for this classification task.

F. Expense Forecasting

FinMate estimates future expenditure using Ridge Regression with temporal features derived strictly from historical observations.

The feature construction includes previous monthly expenditure values, three-month rolling averages, and month information. The use of lagged features prevents future observations from being introduced into the feature vector for a prediction period.

The forecasting process can be represented as

$$\hat{y}_{t+1} = f(y_t, y_{t-1}, y_{t-2}, \bar{y}_{t-2:t}, m_{t+1}), \tag{8}$$

where $\hat{y}_{t+1}$ is the predicted expenditure, y_t represents historical expenditure, $\bar{y}$ represents a rolling average, and m represents the corresponding month information.

TimeSeriesSplit is used during model development to preserve temporal ordering. The implemented system reports a holdout mean absolute error (MAE) of INR 3,369.93. The forecasting component also produces a point estimate together with a 90 percent uncertainty interval.

G. Anomaly Detection

FinMate applies Isolation Forest for unsupervised expenditure anomaly detection. The model is configured with an empirical contamination value of 0.05.

For each transaction or expenditure observation, the model evaluates whether its characteristics are consistent with the learned distribution. Observations identified as unusual are surfaced as analytical signals rather than being automatically treated as fraudulent transactions.

This distinction is important because an expenditure anomaly may represent legitimate user behavior, a billing irregularity, a classification error, or another unusual financial event.

H. Regulatory Knowledge Retrieval

The regulatory RAG subsystem provides domain-specific knowledge to the decision-support pipeline. Its corpus contains financial guidance associated with RBI, SEBI, and FPSB India.

The retrieval process converts the query and indexed knowledge entries into vector representations and compares them using cosine similarity. Relevant entries are then supplied as contextual evidence to the downstream language model.

The cosine similarity between two vectors a and b is calculated as

$$\mathrm{sim}(a, b) = \frac{a \cdot b}{\|a\|\|b\|}. \tag{9}$$

The implemented RAG subsystem reports a Hit Rate at three retrieved results of 93.3 percent and an average retrieval latency of 0.95 ms.

I. Multi-Agent Decision Processing

After financial information and analytical signals have been obtained, the multi-agent subsystem determines which specialized financial agents should participate in the response.

The orchestrator performs intent triage and routes the request to one or more domain specialists. These specialists include transaction, budget, goal, and investment agents. Their outputs are subsequently synthesized by the Financial Decision Agent.

The decision pipeline can be represented as

$$Q \rightarrow O \rightarrow \{A_1, A_2, \ldots, A_n\} \rightarrow E \rightarrow D, \tag{10}$$

where Q represents the financial query, O represents the orchestrator, A_i represents a specialist agent, E represents the collected evidence, and D represents the synthesized decision-support output.

Agent execution is bounded by predefined limits on the number of steps, tool calls, and agent handoffs. These constraints reduce uncontrolled execution and make agent behavior more predictable.

J. Decision Simulation

FinMate provides an in-memory counterfactual simulation mechanism for financial what-if analysis. Instead of modifying persistent financial records, the engine creates a temporary representation of the proposed change and calculates the resulting financial state.

The baseline state contains variables such as income, fixed expenditure, discretionary expenditure, liquid savings, monthly surplus, and financial buffer.

For a scenario with financial delta Δ, the simulated state can be represented as

$$S_{\text{scenario}} = S_{\text{baseline}} + \Delta. \tag{11}$$

The simulation engine evaluates multiple pathways and compares their effects on savings, financial runway, and goal timelines.

Because the calculations remain in memory, exploratory scenarios cannot modify the persistent financial ledger.

K. Language Model Integration

FinMate uses an abstraction layer represented by the `BaseLLMProvider` interface. The architecture supports a deterministic mock provider for testing as well as provider implementations for OpenAI and Google Gemini.

The LLM receives structured evidence generated by backend services. This includes financial calculations, predictions, retrieved regulatory content, and specialist outputs.

The model is therefore primarily responsible for natural language explanation and evidence synthesis. Authoritative numerical values remain under the control of deterministic backend services.

L. Human-in-the-Loop Governance

FinMate distinguishes three classes of system actions:

1) **Read-only operations:** Retrieve or analyze existing information without changing persistent state.
2) **Simulation operations:** Execute temporary what-if calculations in memory without modifying persistent records.
3) **Mutating operations:** Propose changes that would modify the financial ledger or other persistent state.

Mutating operations are staged as pending approval records. The approval mechanism uses a 15-minute expiration window and requires explicit user confirmation before the proposed mutation is committed.

This creates a state transition of the form

$$\text{Proposed} \rightarrow \text{Pending Approval} \rightarrow \begin{cases} \text{Committed}, & \text{if approved,} \\ \text{Rejected}, & \text{if declined or expired.} \end{cases} \tag{12}$$

Consequently, the multi-agent system remains a decision-support mechanism rather than an autonomous financial actor.

M. Android SMS Processing Method

The FinMateSMS client implements a local transaction extraction pipeline. An incoming SMS is first checked against a configured financial-institution sender whitelist. Messages from non-authorized senders are discarded.

The remaining messages are checked for OTP and secret-PIN patterns. Such messages are immediately discarded to prevent sensitive authentication information from entering the transaction pipeline.

Eligible transaction messages are processed by deterministic regular expressions to extract transaction amount, date, and debit or credit type. A SHA-256 hash is generated for duplicate identification, and the structured transaction is stored in the local Room database.

WorkManager subsequently schedules synchronization when network connectivity is available. The backend validates the received payload, performs duplicate verification, and atomically inserts the transaction when it is not already present.

The resulting workflow is

$$SMS \rightarrow Filter \rightarrow Parse \rightarrow Hash \rightarrow Local\ Queue \rightarrow Sync \rightarrow Backend\ Validation \rightarrow Ledger. \tag{13}$$

The raw SMS body is not transmitted as part of this process.

N. Implementation Environment

The backend is implemented using Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2.0, and Alembic. PostgreSQL 16 is used as the primary database, with SQLite available as a fallback. Machine learning components use Scikit-learn together with NumPy and SciPy.

The web application uses React 19, TypeScript, Vite, Tailwind CSS v4, Lucide React, and Recharts. The Android client uses Kotlin 1.9+, AndroidX Room 2.6.1, WorkManager 2.9.0, Retrofit 2.11.0, and OkHttp 4.12.0. Docker and Docker Compose are used for containerized deployment.

The complete methodology therefore forms a pipeline from heterogeneous financial data acquisition through validation, deterministic computation, predictive analysis, regulatory retrieval, multi-agent synthesis, and human-governed decision presentation.

VI. Artificial Intelligence and Retrieval Augmented Generation

FinMate combines conventional machine learning, semantic retrieval, large language models, and multi-agent orchestration within a unified financial intelligence subsystem. The design separates predictive analytics and numerical computation from natural language generation so that generated explanations remain grounded in structured financial evidence.

A. Artificial Intelligence Pipeline

The AI subsystem consists of four principal components:

1) supervised transaction classification;
2) time-series expenditure forecasting;
3) unsupervised expenditure anomaly detection; and
4) regulatory retrieval followed by language model synthesis.

These components operate as analytical services rather than allowing the language model to independently generate financial facts.

The overall intelligence pipeline can be expressed as

$$X \rightarrow \{C(X), F(X), A(X), R(Q)\} \rightarrow E \rightarrow L(E), \tag{14}$$

where X represents financial data, C represents transaction classification, F represents forecasting, A represents anomaly detection, R represents regulatory retrieval, E represents the structured evidence produced by these components, and L represents language-model-based explanation.

B. Transaction Classification Model

FinMate uses a word-level Term Frequency-Inverse Document Frequency (TF-IDF) representation followed by a Linear Support Vector Classifier (LinearSVC) for transaction classification.

For a transaction description d, the TF-IDF representation assigns a weight to each term according to its frequency within the description and its relative frequency across the training corpus. The resulting vector is supplied to the LinearSVC classifier.

The classifier predicts one of ten predefined financial categories:

- Housing
- Food and Dining
- Transportation
- Utilities
- Healthcare
- Entertainment

- Shopping
- Personal Care
- Education
- Investments

The implementation reports a classification accuracy of 98.4 percent and a macro F1-score of 0.9849.

The macro F1-score is particularly useful for evaluating multi-class financial categorization because it gives equal importance to each category rather than allowing frequently occurring transaction classes to dominate the aggregate metric.

C. Expense Forecasting Model

Monthly expenditure forecasting is implemented using Ridge Regression. The forecasting model uses strictly lagged temporal features to reduce the possibility of future information entering the prediction process.

The feature set contains:

- previous-month expenditure;
- second and third monthly lag values;
- three-month rolling expenditure average; and
- month information.

For a prediction at time t, the model therefore depends only on information available before the prediction point.

The Ridge objective can be expressed as

$$\min_{\beta} \left[\sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \lambda \|\beta\|_2^2 \right], \tag{15}$$

where y_i represents observed expenditure, x_i represents the feature vector, β represents the learned coefficients, and λ controls the regularization strength.

Model development uses temporal cross-validation through `TimeSeriesSplit`. The implemented model reports a holdout Mean Absolute Error (MAE) of INR 3,369.93.

The forecasting service also produces a 90 percent uncertainty interval around the expected expenditure value. This enables the downstream decision-support system to present both an expected value and a range of possible expenditure outcomes.

D. Anomaly Detection

FinMate uses Isolation Forest as an unsupervised anomaly detection algorithm. The model identifies observations that differ from the learned distribution of financial transactions.

The implementation uses an empirical contamination parameter of 0.05. The output is treated as an anomaly signal rather than as a definitive fraud classification.

This distinction is important because a statistical expenditure anomaly may have multiple explanations. For example, an unusually large transaction could correspond to a legitimate purchase, an exceptional bill, a duplicate record, or an incorrectly classified transaction.

Accordingly, anomaly results are surfaced to the decision-support layer for contextual analysis rather than triggering autonomous financial actions.

E. Retrieval Augmented Generation

FinMate incorporates Retrieval-Augmented Generation (RAG) to provide domain-specific financial knowledge to the language model.

The RAG subsystem contains a regulatory knowledge corpus associated with:

- Reserve Bank of India (RBI) guidance;
- Securities and Exchange Board of India (SEBI) guidance; and
- Financial Planning Standards Board India (FPSB India) guidance.

The retrieval mechanism represents knowledge entries and incoming queries in a vector space and calculates semantic similarity using cosine similarity.

For query vector q and document vector d, the similarity is

$$\cos(q, d) = \frac{q \cdot d}{\|q\| \|d\|}. \tag{16}$$

Documents are ranked according to their similarity to the query, and the highest-ranking entries are selected as contextual evidence.

The implemented retrieval system reports a Hit Rate at three results of 93.3 percent and an average retrieval latency of 0.95 ms.

F. Regulatory Grounding

The retrieval component provides an evidence layer between the user's financial question and language model generation.

Instead of allowing the LLM to independently determine regulatory information, the system first retrieves relevant knowledge entries. These entries are then provided as contextual information for response generation.

The conceptual process is

$$Q \rightarrow \text{Retrieve}(Q, K) \rightarrow K_r \rightarrow \text{Generate}(Q, K_r), \tag{17}$$

where Q represents the user query, K represents the regulatory knowledge corpus, K_r represents retrieved knowledge, and Generate represents the language model response stage.

This architecture allows regulatory information to be separated from free-form language generation and provides a mechanism for presenting retrieved evidence alongside financial explanations.

G. Large Language Model Abstraction

FinMate does not directly couple the application to a single language model provider. Instead, the system defines a common `BaseLLMProvider` abstraction.

The implementation supports:

- `MockLLMProvider` for deterministic and low-cost testing;
- `OpenAILLMProvider` for OpenAI-based language models; and
- `GeminiLLMProvider` for Google Gemini-based language models.

The abstraction allows the orchestration layer to remain independent of the underlying provider.

The language model receives structured information generated by trusted backend components. Depending on the query, this information may include transaction summaries, financial calculations, forecasts, anomaly signals, scenario results, and retrieved regulatory knowledge.

H. Separation of Computation and Generation

A central design principle of FinMate is the separation of financial computation from language generation.

The language model is not treated as the authoritative source for monetary values. Instead, numerical results are produced by deterministic financial services or trained analytical models.

The architecture can therefore be represented as

$$D_{\text{financial}} \rightarrow C_{\text{deterministic}} \rightarrow E_{\text{structured}} \rightarrow L_{\text{narrative}}, \quad (18)$$

where $D_{\text{financial}}$ represents financial data, $C_{\text{deterministic}}$ represents deterministic computation, $E_{\text{structured}}$ represents structured evidence, and $L_{\text{narrative}}$ represents natural language explanation.

This design reduces the possibility that the language model will independently invent or alter monetary values during response generation.

I. Multi-Agent Intelligence

The language model subsystem is integrated with FinMate's multi-agent architecture through a centralized orchestrator.

The orchestrator performs intent triage and determines which specialized agents should participate in the requested task. The principal agents are the Transaction Specialist, Budget Specialist, Goal Specialist, Investment Specialist, and Financial Decision Agent.

A simple execution model is

$$Q \rightarrow O \rightarrow \{A_T, A_B, A_G, A_I\} \rightarrow D_A, \quad (19)$$

where Q is the user query, O is the orchestrator, A_T, A_B, A_G, and A_I represent the transaction, budget, goal, and investment specialists respectively, and D_A represents the final decision synthesis stage.

Not every query requires every specialist. The orchestrator selects an appropriate execution route based on the detected intent and available financial evidence.

J. Decision Synthesis

The Financial Decision Agent combines outputs from the participating specialists with deterministic financial calculations and retrieved knowledge.

Rather than returning a single opaque recommendation, the decision layer can construct multiple decision alternatives and expose associated trade-offs.

For example, a financial scenario may be represented as

$$D = \{O_A, O_B, O_C\}, \quad (20)$$

where each O_i represents a distinct scenario containing its financial impact, assumptions, and trade-offs.

This structure is intended to preserve user agency by presenting financial alternatives rather than allowing an autonomous agent to execute financial actions.

K. Prompt Injection Detection

FinMate includes a prompt-injection detection mechanism before orchestration execution. Incoming instructions are inspected for patterns associated with attempts to override system constraints, manipulate tool execution, or bypass agent governance.

Detected adversarial inputs are rejected before the agent is permitted to invoke governed tools.

This security boundary is particularly important because the multi-agent architecture has access to structured financial information and state-changing proposal mechanisms.

L. Agent Execution Constraints

The orchestrator enforces explicit execution boundaries:

- maximum agent steps: 8;
- maximum tool calls: 12;
- maximum agent handoffs: 3.

These constraints limit execution depth and reduce the risk of uncontrolled agent loops.

Tool access is additionally restricted through per-agent allowlists. Consequently, a specialist cannot arbitrarily invoke tools belonging to another financial domain.

M. AI Decision Support Pipeline

The complete AI pipeline can therefore be summarized as:

1) The user submits a financial question or scenario.
2) The FastAPI gateway validates the request.
3) The orchestrator performs intent triage.
4) Relevant specialist agents are selected.
5) Agents retrieve structured financial information through governed tools.
6) Machine learning services provide classification, forecast, or anomaly outputs where applicable.
7) The RAG subsystem retrieves relevant regulatory information.
8) Deterministic financial services calculate authoritative monetary values.
9) The Financial Decision Agent synthesizes the collected evidence.
10) The LLM converts the structured evidence into a natural language explanation.
11) If the requested operation would modify persistent state, the result is passed to the human-in-the-loop approval mechanism rather than being committed automatically.

This architecture allows FinMate to combine the flexibility of generative AI with deterministic financial computation, statistical machine learning, regulatory retrieval, and controlled agent execution.

VII. Security and Privacy Architecture

Security and privacy are core design requirements of FinMate because the system processes personal financial information and provides intelligent decision support over that information. The security architecture is implemented across the data ingestion, API, agent, language model, and state mutation layers.

A. Security Design Principles

The FinMate security architecture follows four principal design principles:

1) minimize the exposure of sensitive financial information;
2) restrict agent capabilities through explicit tool permissions;
3) separate analysis and simulation from persistent state mutation;
4) require explicit human authorization for state-changing actions.

These principles are implemented through multiple independent controls rather than relying on a single security mechanism.

B. Human-in-the-Loop State Governance

FinMate does not operate as an autonomous financial transaction agent. Actions are classified into read-only, simulation, and mutating categories.

Read-only actions can retrieve and analyze existing financial information. Simulation operations execute temporary calculations in memory. Mutating operations are treated differently because they could alter persistent financial records.

A mutating operation is therefore placed into a pending approval state rather than being immediately committed. The approval record has a 15-minute expiration window. The operation can only transition to a committed state after explicit user confirmation.

The state transition is represented as

$$P \rightarrow \begin{cases} C, & \text{explicit approval}, \\ R, & \text{explicit rejection}, \\ E, & \text{expiration}, \end{cases} \tag{21}$$

where P denotes a pending operation, C denotes a committed operation, R denotes rejection, and E denotes expiration.

This mechanism ensures that an agent cannot silently modify the user's financial ledger.

C. Governed Agent Tool Access

The multi-agent architecture uses a governed registry containing fifteen tools. Each tool has a defined input and output contract enforced through Pydantic schemas.

Agents are assigned tool allowlists according to their domain. For example, a transaction specialist is restricted to transaction-related operations, while a goal specialist receives access to goal-related operations.

The architecture therefore limits the capability of individual agents and reduces the consequences of unintended or malicious tool invocation.

In addition to tool allowlists, execution boundaries are enforced through maximum agent steps, tool calls, and handoffs. The implemented limits are eight agent steps, twelve tool calls, and three agent handoffs.

D. Tenant Isolation

Financial records are associated with individual user contexts. The governed tool registry applies tenant injection when domain operations are executed.

Consequently, a tool invocation is associated with the authenticated financial context rather than allowing an agent to freely specify an arbitrary user identifier.

This mechanism provides an additional isolation layer between agent execution and persistent financial data.

E. Prompt Injection Defense

FinMate includes a prompt-injection detection mechanism before orchestration execution. Incoming instructions are inspected for patterns associated with attempts to override system instructions, manipulate agent behavior, or bypass tool restrictions.

When an adversarial instruction is detected, the request is rejected before the agent is allowed to execute governed tools.

This creates a security boundary between untrusted natural language input and privileged financial operations.

F. Deterministic Computation Boundary

Financial calculations are isolated from language model generation. Monetary operations use Python `Decimal` arithmetic with `ROUND_HALF_UP` and database `NUMERIC(15,2)` fields.

This design reduces the possibility that an LLM-generated numerical value will directly become an authoritative financial value.

The language model instead receives structured outputs generated by deterministic financial services and analytical models. It is therefore used primarily for explanation and synthesis.

G. Android SMS Privacy Model

The FinMateSMS component is designed around local processing of bank SMS alerts.

Incoming messages pass through a sender whitelist. Only messages associated with recognized financial institution sender identifiers are processed.

Messages are then checked for OTP and secret-PIN patterns. Messages identified as authentication or secret-bearing messages are immediately discarded.

Eligible transaction alerts are parsed locally using deterministic regular expressions. The parser extracts only the structured information required for transaction ingestion.

The resulting architecture is

$$SMS \rightarrow Whitelist \rightarrow Secret\ Filter \rightarrow Local\ Parser \rightarrow Structured\ Transaction \tag{22}$$

The raw SMS body is not transmitted to the backend.

H. Local Transaction Protection

Parsed SMS transactions are stored locally using AndroidX Room. A SHA-256 hash is generated to provide a stable transaction reference.

Room applies a unique constraint to the hash, preventing duplicate local entries. During synchronization, the backend performs an independent duplicate check using the same source reference.

If the transaction already exists on the server, the backend returns a conflict response rather than creating another ledger entry.

This dual-layer design reduces duplicate insertion caused by retries, repeated SMS processing, or intermittent network connectivity.

I. Secure Data Transmission

The Android client synchronizes structured transaction data through the versioned FastAPI API. Retrofit and OkHttp provide the networking layer.

The synchronization payload contains structured transaction information rather than the original SMS message. Backend Pydantic schemas validate the received data before it can enter the financial ledger.

The backend subsequently performs duplicate verification and atomic transaction insertion.

J. Data Lineage and Auditability

FinMate maintains transaction lineage through source type, source reference, import identifier, and data-quality flags.

Agent-related activity is additionally represented through agent task, approval, and tool-call records.

This provides an auditable representation of how financial information entered the system and how governed agent operations interacted with financial data.

K. Security Through Architectural Separation

The security architecture does not depend on a single protective layer. Instead, FinMate combines:

- local SMS parsing;
- sender whitelisting;
- OTP and secret-PIN filtering;
- SHA-256 source hashing;
- local and server-side duplicate prevention;
- structured API validation;
- tenant injection;
- agent-specific tool allowlists;
- bounded agent execution;
- prompt-injection detection;
- deterministic financial computation;
- human approval for state-changing operations.

This layered approach is intended to reduce privacy exposure, restrict agent capabilities, and preserve human control over persistent financial state.

VIII. Experimental Evaluation and Results

The implemented FinMate system was evaluated across machine learning, retrieval, multi-agent behavior, automated software testing, and frontend build validation. The evaluation results reported in the project documentation are summarized in this section.

A. Evaluation Dimensions

The evaluation covers the following system dimensions:

1) transaction classification performance;
2) expenditure forecasting performance;
3) regulatory retrieval performance;
4) multi-agent benchmark performance;
5) automated unit and integration testing;
6) prompt-injection defense behavior; and
7) production frontend build validation.

The evaluation is intended to assess both the individual analytical components and the behavior of the integrated application.

B. Transaction Classification Evaluation

The transaction classification subsystem uses a TF-IDF representation with a LinearSVC classifier.

The implemented system reports an accuracy of 98.4 percent and a macro F1-score of 0.9849 across the ten financial transaction categories.

TABLE I
Transaction Classification Results

Metric	Result
Accuracy	98.4%
Macro F1-score	0.9849
Number of categories	10
Classifier	LinearSVC
Feature representation	TF-IDF

The macro F1-score indicates the average classification quality across the defined categories without weighting categories according to their frequency.

C. Expense Forecasting Evaluation

The expense forecasting subsystem uses Ridge Regression with strictly lagged temporal features.

Model development uses temporal cross-validation to preserve the chronological ordering of financial observations. The implemented system reports a holdout Mean Absolute Error of INR 3,369.93.

The reported error should be interpreted together with the underlying dataset characteristics. In the final experimental version of the paper, the dataset size, temporal coverage, baseline models, and evaluation protocol should be explicitly documented.

TABLE II
EXPENSE FORECASTING RESULT

Metric	Result
Model	Ridge Regression
Validation strategy	TimeSeriesSplit
Holdout MAE	INR 3,369.93
Forecast interval	90%

D. Regulatory Retrieval Evaluation

The RAG subsystem was evaluated using retrieval hit rate and retrieval latency.

The implemented system reports a Hit Rate at three retrieved results of 93.3 percent. The average retrieval latency reported by the project is 0.95 ms.

TABLE III
REGULATORY RETRIEVAL RESULTS

Metric	Result
Hit Rate at 3	93.3%
Average retrieval latency	0.95 ms
Similarity metric	Cosine similarity
Knowledge sources	RBI, SEBI, FPSB India

The retrieval evaluation measures the ability of the semantic retrieval component to return relevant entries from the financial knowledge corpus. It does not independently measure the quality of the subsequent language model generation.

E. Multi-Agent Benchmark

The project documentation reports a benchmark containing twelve standard financial agent tasks.

The implemented system successfully completed all twelve benchmark scenarios, resulting in a reported success rate of 12 out of 12.

The benchmark covers individual specialist requests, cross-specialist deliberation, approval-related behavior, and security-related scenarios.

An additional invariant checks that numerical outputs remain consistent with backend financial records. Prompt-injection scenarios are also tested to ensure that adversarial inputs are rejected before governed tools are executed.

TABLE IV
MULTI-AGENT BENCHMARK RESULT

Metric	Result
Benchmark tasks	12
Successful tasks	12
Reported success rate	100%
Numerical consistency check	Passed
Prompt-injection defense	Passed

F. Automated Software Testing

FinMate contains automated unit and integration tests covering the principal backend components and APIs.

The project reports 102 out of 102 tests passing, corresponding to a 100 percent reported test pass rate.

The test suite includes validation of:

- financial calculations;
- transaction classification;
- forecasting;
- anomaly detection;
- RAG retrieval;
- multi-agent orchestration;
- approval governance;
- tool registry behavior;
- CSV parsing and normalization;
- data-quality rules;
- decision simulation;
- language-model guardrails; and
- REST API integration.

TABLE V
AUTOMATED TEST RESULTS

Metric	Result
Total automated tests	102
Passed tests	102
Failed tests	0
Reported pass rate	100%

G. Frontend Build Evaluation

The production frontend is built using TypeScript and Vite.

The project documentation reports that the production build completed with zero errors and zero warnings across 1,892 transformed modules.

This provides an implementation-level validation of the frontend build pipeline in addition to the backend automated test suite.

H. Overall Evaluation Summary

Table VI summarizes the principal reported evaluation results.

TABLE VI
OVERALL FINMATE EVALUATION RESULTS

Evaluation Component	Reported Result
Transaction accuracy	98.4%
Transaction macro F1	0.9849
Forecast holdout MAE	INR 3,369.93
RAG Hit Rate at 3	93.3%
RAG latency	0.95 ms
Agent benchmark	12/12
Automated tests	102/102
Frontend build errors	0
Frontend build warnings	0

These results indicate that the implemented system successfully integrates the individual analytical and software components. However, the reported metrics should not be interpreted as evidence of universal financial prediction or decision-making performance. Model performance depends on

the datasets, evaluation procedures, and deployment environment.

I. Reproducibility Considerations

For final publication, the experimental section should document the datasets used for model training and evaluation, dataset sizes, train-test partitions, feature construction, hardware configuration, software versions, and baseline comparisons.

This additional information is necessary to enable independent reproduction of the reported machine learning and retrieval results.

IX. Discussion and Limitations

The implementation of FinMate demonstrates how deterministic financial services, machine learning, retrieval augmented generation, and multi-agent orchestration can be integrated into a single personal financial decision-support architecture.

A. Integration of Heterogeneous Intelligence

A central characteristic of FinMate is that different computational approaches are assigned different responsibilities.

Machine learning models perform specialized predictive tasks such as transaction classification, expenditure forecasting, and anomaly detection. The deterministic financial engine performs authoritative monetary calculations. The RAG subsystem retrieves domain-specific knowledge, while the language model provides natural language explanation and synthesis.

This division avoids treating the language model as a replacement for every computational component. Instead, the LLM operates as an interpretation and synthesis layer over structured evidence.

B. Deterministic Financial Computation

The separation of financial calculations from language generation is an important architectural property.

Financial values such as surplus, savings rates, required goal contributions, and scenario changes are calculated by backend services using decimal arithmetic. This provides a deterministic source of numerical values that can be independently verified.

The language model can therefore explain the result without becoming the authority responsible for calculating the underlying monetary quantity.

C. Multi-Agent Governance

The multi-agent architecture provides specialization by assigning financial domains to separate agents. The orchestrator coordinates these agents while enforcing execution boundaries and tool permissions.

This approach provides a structured mechanism for handling cross-domain financial queries. However, multi-agent execution also introduces additional complexity, including routing errors, tool misuse, repeated execution, and inconsistent intermediate outputs.

FinMate addresses these concerns through bounded execution, tool allowlists, structured schemas, and human approval for state-changing operations. These mechanisms do not eliminate all agent-related risks, but they establish explicit control boundaries.

D. Regulatory Grounding

The RAG subsystem provides a mechanism for grounding financial explanations in a dedicated regulatory knowledge corpus.

The use of retrieved evidence is particularly relevant when a financial question depends on external guidance. However, retrieval quality is dependent on the completeness, correctness, and maintenance of the underlying corpus.

A retrieval system can also return contextually similar information that does not fully answer a user's question. Consequently, retrieval performance should be considered separately from the correctness of the final generated explanation.

E. Privacy-Oriented Transaction Ingestion

The Android SMS ingestion architecture reduces privacy exposure by performing message parsing locally. Raw SMS content is not transmitted to the backend.

Sender whitelisting and OTP or secret-PIN filtering further constrain which messages enter the transaction pipeline. Local and server-side hash-based duplicate protection provides resilience against repeated processing.

Nevertheless, local SMS ingestion requires appropriate Android permissions and depends on the structure of bank-generated transaction alerts. Changes in sender identifiers or message formats may require updates to the filtering and parsing rules.

F. Human Agency and State Mutation

FinMate intentionally distinguishes between decision support and autonomous financial action.

Read-only analytical operations can execute automatically, while simulations operate in memory. Operations that would change persistent financial state require explicit human approval.

This design ensures that the system presents financial information and possible actions without independently executing consequential financial operations.

G. System-Level Evaluation

The implemented system reports strong results across its internal evaluation suite, including 98.4 percent transaction classification accuracy, a 0.9849 macro F1-score, 93.3 percent RAG Hit Rate at three, 102 out of 102 automated tests passing, and 12 out of 12 agent benchmark tasks passing.

These results demonstrate successful implementation and internal validation of the corresponding components. They should not, however, be interpreted as evidence that FinMate will achieve identical performance on all users, financial institutions, datasets, or deployment environments.

H. Limitations

Several limitations remain.

First, the reported machine learning metrics are dependent on the datasets used during development and evaluation. Broader datasets covering different transaction descriptions, spending patterns, and user populations are required to establish external generalization.

Second, the forecasting error is reported as a single holdout metric. Additional baselines and evaluation periods would provide a stronger assessment of forecasting performance.

Third, the RAG evaluation measures retrieval performance rather than the complete end-to-end correctness of generated financial explanations.

Fourth, the multi-agent benchmark contains twelve scenarios. Although the benchmark provides useful implementation validation, larger and more diverse evaluation suites are required to assess general agent behavior.

Finally, the current architecture is a decision-support system and does not independently execute financial transactions, fund transfers, or investment orders. Its purpose is to support informed human decision making rather than replace it.

I. Implications for Deployment

The architecture suggests that financial AI systems can benefit from explicit separation between prediction, computation, retrieval, generation, and state mutation.

Such separation enables each subsystem to be evaluated independently and allows security controls to be placed at the boundaries between untrusted input, analytical services, language generation, and persistent state.

For FinMate, this architecture provides a foundation for future experimentation with larger datasets, alternative predictive models, improved retrieval strategies, and more capable language models without requiring the core financial computation and governance mechanisms to be replaced.

X. Future Work

Although FinMate integrates machine learning, retrieval augmented generation, multi-agent orchestration, deterministic financial computation, and privacy focused transaction ingestion, several areas remain open for further development.

A. Improved Financial Forecasting

The current expenditure forecasting component uses Ridge Regression with strictly lagged features and time-series validation. Future versions can evaluate additional forecasting models, including gradient boosting, probabilistic forecasting, and sequence-based models. Model selection can also be performed adaptively according to the amount and regularity of historical transaction data available for an individual user.

Future evaluation should additionally consider forecasting performance across different income patterns, spending frequencies, and transaction volumes rather than relying only on aggregate error measures.

B. Advanced Retrieval and Knowledge Grounding

The current regulatory retrieval pipeline uses TF-IDF representations and cosine similarity over a curated financial knowledge base. Future work can investigate dense embeddings, hybrid sparse–dense retrieval, reranking, and document-level citation tracking.

An expanded knowledge base could incorporate additional authoritative financial guidance and maintain document versions so that responses can be associated with the regulatory information available at a particular point in time. This would be particularly useful when regulations or financial guidance are updated.

C. Personalized Financial Decision Support

FinMate currently provides financial analysis based on transaction history, goals, budgets, forecasts, and retrieved knowledge. Future work can extend the personalization layer by learning longer-term user preferences and financial behavior while maintaining explicit privacy controls.

Personalization should remain bounded by the deterministic financial engine and governance layer. In particular, personalization should influence the presentation and prioritization of recommendations rather than allowing a language model to independently modify financial records or execute financial transactions.

D. Extended Multi-Agent Coordination

The current architecture contains specialized agents for transaction analysis, budgeting, goals, investments, and financial decision synthesis. Future research can investigate more sophisticated coordination strategies, including adaptive task decomposition, agent confidence estimation, and cross-agent verification.

Additional evaluation can measure whether multi-agent decomposition provides measurable improvements over equivalent single-agent workflows in terms of accuracy, latency, reliability, and tool-use efficiency.

E. Privacy-Preserving Mobile Intelligence

The Android client currently performs transaction extraction locally from authorized bank SMS messages and avoids transmitting raw SMS content. Future versions can further strengthen this architecture through additional on-device processing, encrypted local storage, improved sender classification, and more extensive privacy auditing.

A useful direction is to evaluate the complete data lifecycle from SMS capture through local parsing, API transmission, server-side persistence, analysis, and deletion. Such evaluation can provide stronger evidence that privacy guarantees are preserved across the complete system rather than at only the ingestion stage.

F. Explainability and Decision Provenance

Future versions can provide explicit provenance for financial decisions by linking each generated explanation to the numerical calculations, machine learning outputs, retrieved regulatory sources, and agent actions that contributed to it.

A provenance graph could allow users to inspect why a particular financial insight was generated and distinguish deterministic results from model predictions and language-model explanations. This would improve transparency and facilitate auditing of complex multi-agent workflows.

G. Larger-Scale Evaluation

The current evaluation focuses on component-level metrics, automated tests, and agent benchmark tasks. Future work should evaluate FinMate using larger and more diverse datasets and longer-term usage scenarios.

Evaluation can include user studies measuring comprehension, trust, decision quality, interaction effort, and perceived usefulness. System-level experiments can additionally measure latency, resource consumption, failure recovery, and scalability under concurrent workloads.

H. Deployment and Observability

The current implementation provides containerized backend deployment and production-oriented build artifacts. Future work can extend operational observability through structured telemetry, distributed tracing, model monitoring, retrieval monitoring, and automated anomaly detection for system behavior.

Such capabilities would support reliable deployment of the platform at larger scale while preserving the separation between financial computation, AI reasoning, data storage, and governed state mutation.

Overall, future development of FinMate should focus on improving predictive accuracy, knowledge grounding, personalization, privacy, explainability, and scalability without weakening the deterministic and human-governed properties of the system.

XI. Conclusion

This paper presented FinMate, a personal financial decision support system that integrates deterministic financial computation, machine learning, retrieval augmented generation, multi-agent orchestration, and privacy focused transaction ingestion within a unified architecture.

The system addresses several limitations associated with conventional personal finance applications and unconstrained language-model-based financial assistants. Transaction records are processed using a machine learning classification pipeline, expenditure behavior is analyzed using forecasting and anomaly detection models, and financial guidance is grounded through retrieval from a curated regulatory knowledge base. Specialized agents coordinate these capabilities through a centralized orchestration layer and a governed tool registry.

A central design principle of FinMate is the separation of financial computation from language generation. Monetary values are calculated using deterministic backend logic and Python `Decimal` arithmetic rather than being generated by a language model. This provides a controlled computational boundary while allowing language models to provide natural language interaction and explanation.

The system also incorporates explicit governance for state-changing operations. Human-in-the-loop approval, execution constraints, tool authorization, state validation, and operation expiry are used to prevent the multi-agent system from autonomously performing persistent financial actions. Counterfactual scenarios are similarly evaluated in memory without modifying persistent financial records.

Privacy is addressed at the transaction-ingestion layer through the Android SMS processing pipeline. Transaction alerts can be parsed locally on the device, while raw SMS content is not transmitted to the backend. Additional controls such as sender filtering, OTP and PIN exclusion, hashing, and duplicate prevention provide further protection during ingestion.

The implemented evaluation demonstrates the feasibility of the proposed architecture. The transaction classifier reports 98.4 percent accuracy and a macro F1 score of 0.9849. The regulatory retrieval component reports a 93.3 percent Hit Rate@3 with a measured retrieval latency of 0.95 ms. The expense forecasting component reports a holdout mean absolute error of INR 3,369.93, while the multi-agent benchmark completed 12 out of 12 benchmark tasks. The automated backend test suite reports 102 out of 102 tests passing, and the frontend production build completes without errors or warnings.

These results indicate that a personal financial assistant can combine conversational AI with deterministic computation, domain-grounded retrieval, machine learning, privacy-oriented data ingestion, and human governance in a single system. FinMate therefore provides an engineering framework for building financial decision-support applications in which AI assists the user while numerical correctness, data protection, and control over state-changing operations remain explicit system properties.

References

[1] S. Vaduka, S. C. Reddy, H. V. Arikathota, L. Konchada, P. Chakraborty, and A. Bandyopadhyay, "Optimizing personal finance management through ai-driven decision support systems," in *2024 IEEE Region 10 Symposium (TENSYMP)*, 2024.

[2] N. Shailendra, C. Parvathi, S. Gupta, and G. Ravikumar, "Money map: The personal finance tracker," in *2025 International Conference on Knowledge Engineering and Communication Systems (ICKECS)*, 2025.

[3] H. N. Lakshmi, C. V. L. Narayana, J. Krishna, A. Vishalakshi, S. Swama, and B. V. Sivaiah, "Ai-powered financial consultant for secure and intelligent personal finance management," in *2026 IEEE International Conference on Emerging Computing and Intelligent Technologies (ICoECIT)*, 2026.

[4] A. Alhenshiri, R. Shgaibe, and E. Abied, "An intelligent system for semantic classification of personal expenses from transaction descriptions," *The International Journal of Engineering & Information Technology*, vol. 14, no. 2, pp. 146–152, 2026.

[5] "Online transaction anomaly detection model for credit card usage using machine learning classifiers," in *2023 International Conference on Emerging Smart Computing and Informatics (ESCI)*, 2023.

[6] "Hybrid isolation forest-based machine learning for fraud detection in digital transactions," in *2026 6th International Conference on Intelligent Technologies (CONIT)*, 2026.

[7] Z. Yang, B. Biibosunov, S. Biibosunova, G. Kalmanbetova, and X. Wang, "A deep learning framework for automated transaction classification and budget prediction in personal financial management," *Discover Artificial Intelligence*, vol. 6, 2026.

[8] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, S. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *arXiv preprint arXiv:2005.11401*, 2020.

[9] "A survey of retrieval-augmented generation (rag) for large language models," in *2025 International Conference on Trustworthy Big Data and Artificial Intelligence (ICTBAI)*, 2025.

[10] D. Maldonado, E. Cruz, J. Abad Torres, and P. J. Cruz, "Multi-agent systems: A survey about its components, framework and workflow," *IEEE Access*, vol. 12, pp. 80 950–80 975, 2024.

[11] Y. Wang, Y. Pan, Z. Su, Y. Deng, Q. Zhao, L. Du, T. H. Luan, J. Kang, and D. Niyato, "Large model-based agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1906–1949, 2026.

[12] D. Lee and H. Kim, "A survey on llm agents: Architecture, applications, and challenges," in *2026 40th International Conference on Information Networking (ICOIN)*, 2026, pp. 979–982.